

Les Bases du Langage

JAVASCRIPT



Les bases : Syntaxe et Nommage

Les instructions :

- Les points-virgules séparent les instructions JavaScript.
- Il est recommandé de terminer l'instruction par un point virgule « ; »
- Il est possible d'écrire plusieurs instructions dans la même ligne à condition de les séparer avec un point-virgule.

Les bases : Syntaxe et Nommage

Les espaces blanc :

- Javascript ignore les espace blancs .
- Une bonne pratique consiste a placer des espace autour des operateurs par Exemple : (a = b) au lieu (a=b).

Les bases : Syntaxe et Nommage

Les commentaires :

- `//` commentaire sur une ligne
- `/*` sur plusieurs ligne `*/`

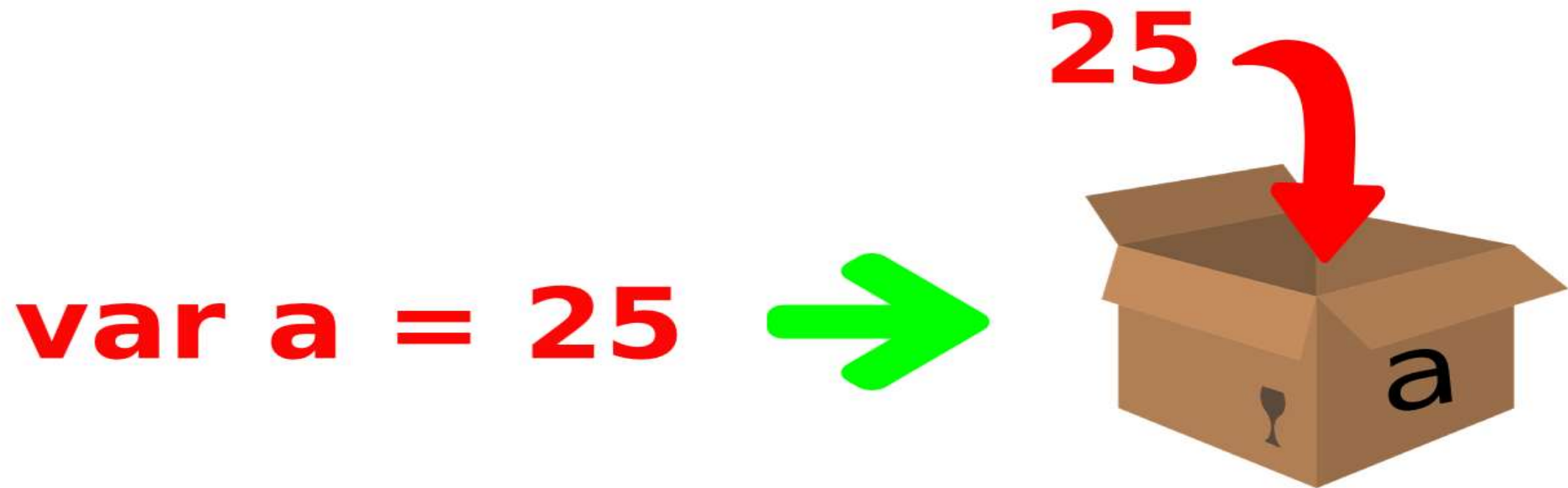
Les bases : Syntaxe et Nommage

Nommage:

- Javascript est sensible a la casse : « name » != « Name ».
- Nom des fonction et variable en camelCase.
- Une variable globale en majuscule : MA_VARIABLE.
- Une variable au pluriel indique généralement un tableau.
- Les trait d'union ne sont pas autorisé (-). Il sont réservé a l'operateur de soustraction.

abstract	arguments	await*	boolean
break	byte	case	catch
char	class*	const	continue
debugger	default	delete	do
double	else	enum*	eval
export*	extends*	false	final
finally	float	for	function
goto	if	implements	import*
in	instanceof	int	interface
let*	long	native	new
null	package	private	protected
public	return	short	static
super*	switch	synchronized	this
throw	throws	transient	true
try	typeof	var	void
volatile	while	with	yield

Les bases : Les variables

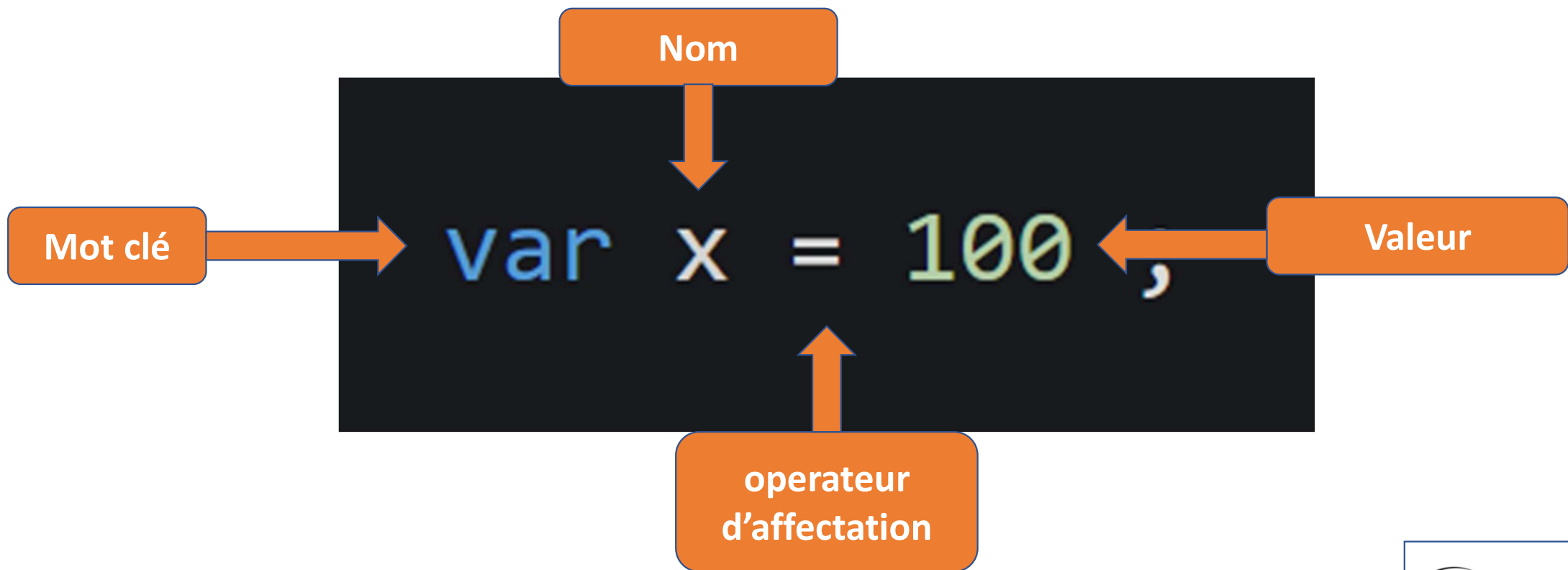


Les bases : Les variables

Règles de nommage des variables :

- Les noms peuvent contenir des lettres, des chiffres, des traits de soulignement et des signes dollar.
- Les noms doivent commencer par : une lettre || signe \$ || Under score _.
- Les noms ne doivent pas commencer par un nombre.
- Les noms ne doivent pas contenir d'espace.
- Les mots réservés ne peuvent pas être utilisés comme noms.
- Sensibilité à la casse («a» et «A» sont des variables différentes).

Les bases : Les variables



Les bases : Les variables

Déclaration et attribution

```
var nom ;
```

Déclaration de variable

```
var nom = "Javascript";
```

Attribution de valeur

Les bases : Les variables

Déclaration plusieurs variables

```
var nom = "Javascript"; var prix = 12;
```

Les bases : Les variables

Déclaration plusieurs lignes

```
var nom = " Joe " ,  
    age = 18 ,  
    hobbies = "Sport"
```

Les bases : Types de données

Données primitives :

Type de données	Description
String	représente une séquence de caractères, par exemple "bonjour"
Number	représente des valeurs numériques, par exemple 100
boolean	représente la valeur booléenne false ou true
undefined	représente une valeur indéfinie
Null	représente nul, c'est-à-dire aucune valeur du tout

Les bases : Types de données

Données non primitives :

Type de données	Description
Object	représente une instance par laquelle nous pouvons accéder aux membres
Array	représente un groupe de valeurs similaires
RegExp	représente une expression régulière

Les bases : Types de données

Données non primitives :

Type de données	Description
Object	représente une instance par laquelle nous pouvons accéder aux membres
Array	représente un groupe de valeurs similaires
RegExp	représente une expression régulière

Les bases : Types de données

Données non primitives :

Type de données	Description
Object	représente une instance par laquelle nous pouvons accéder aux membres
Array	représente un groupe de valeurs similaires
RegExp	représente une expression régulière

Operators in JAVASCRIPT



Les bases : Les operateurs

Opérateurs arithmétiques :

Opérateur	Description	Exemple
+	Une addition	$10+20 = 30$
-	Soustraction	$20-10 = 10$
*	Multiplication	$10*20 = 200$
/	Division	$20/10 = 2$
%	Module (reste)	$20\%10 = 0$
++	Incrément	<code>var a = 10; a ++; Maintenant a = 11</code>
--	Décrémenter	<code>var a = 10; une--; Maintenant a = 9</code>

Les bases : Les operateurs

Opérateurs d'affectation:

Opérateur	Description	Exemple
=	Attribuer	var a = 20
+=	Ajouter et attribuer	var a = 10; a + = 20; Maintenant a = 30
-=	Soustraire et attribuer	var a = 20; a - = 10; Maintenant a = 10
*=	Multiplier et attribuer	var a = 10; a * = 20; Maintenant a = 200
/=	Diviser et attribuer	var a = 10; a / = 2; Maintenant a = 5
%=	Module et attribuer	var a = 10; un% = 2; Maintenant a = 0

Les bases : Les operateurs

Opérateurs de comparaison :

Opérateur	Description	Exemple
==	Est égal à	10 == 20 = faux
===	Identique (égal et de même type)	10 === 20 = faux
!=	Pas égal à	10 != 20 = vrai
!==	Pas identique	20 !== 20 = faux
>	Plus grand que	20 > 10 = vrai
>=	Plus grand ou égal à	20 >= 10 = vrai
<	Moins que	20 < 10 = faux
<=	Inférieur ou égal à	20 <= 10 = faux

Les bases : Les operateurs

Opérateurs logiques:

Opérateur	Description	Exemple
&&	ET logique	<code>(10 == 20 && 20 == 33) = faux</code>
	OU logique	<code>(10 == 20 20 == 33) = faux</code>
!	Pas logique	<code>! (10 == 20) = vrai</code>



Les bases : Les conditions

Expressions conditionnelles:

- Les instructions if
- Les instructions if-else
- Les instructions if-else-if
- Les instructions de commutation Switch

Les bases : Les conditions

Déclaration if :

```
if ( condition ) {  
    code ..  
}
```

```
if ( 7 > 5 ) {  
    console.log( ' Vrai' )  
}
```


Les bases : Les conditions

Déclaration if-else :

```
if ( condition ) {  
    instruction 1;  
} else {  
    instruction 2;  
}
```

```
if ( 7 > 5 ) {  
    console.log( ' Vrai' );  
} else {  
    console.log( 'Faux' );  
}
```

Les bases : Les conditions

Déclaration if-else-if :

```
if ( condition ) {  
    instruction 1;  
} else if ( condition ) {  
    instruction 2;  
} else {  
    instruction 3;  
}
```

```
if ( a > 5 ) {  
    console.log( 'Grand' );  
} else if ( a == 5 ) {  
    console.log( 'Egal' );  
} else {  
    console.log( 'Petit' );  
}
```

Les bases : Les conditions

Déclaration switch case :

```
switch (expression) {  
  case value 1:  
    statement;  
    break;  
  
  case value 2:  
    statement;  
    break;  
  
  default:  
    statement;  
}
```

```
switch (value = 2) {  
  case 1 :  
    console.log( ' Petit ' );  
    break;  
  
  case 2 :  
    console.log( ' Egal ' );  
    break;  
  
  default:  
    console.log( 'Ché pas' );  
}
```

Les bases : Les Fonctions

Déclaration de base:

```
function myFunction (p1, p2) {  
    return p1 * p2;  
}
```

Invocation de fonction

Le code à l'intérieur de la fonction s'exécutera lorsque « quelque chose » **invocuera** (appellera) la fonction :

```
myFunction (2, 2)
```